

Trabalho Prático 1 — Controlador de Cache

1. Introdução

Neste trabalho prático, vocês deverão implementar um controlador de cache conforme descrito no *Capítulo 5, Seção 5.12* do livro *Computer Organization and Design: The Hardware/Software Interface — RISC-V Edition*.

O objetivo é consolidar os conceitos relacionados à hierarquia de memória, com ênfase em:

- funcionamento de caches;
- políticas de leitura e escrita;
- tratamento de *misses*;
- interação entre cache e memória principal.

A implementação deverá ser realizada utilizando a linguagem de descrição de hardware SystemVerilog, tomando como base a especificação apresentada no livro.

2. Objetivos

Ao final deste trabalho, espera-se que o grupo seja capaz de:

- Compreender a arquitetura e o funcionamento de um controlador de cache;
- Implementar, em nível de hardware, os mecanismos de acesso e controle de cache;
- Validar funcionalmente o projeto por meio de simulação;
- Organizar um projeto de hardware de forma reprodutível e documentada.

3. Requisitos do Projeto

O projeto deverá contemplar:

- Implementação completa do controlador de cache conforme especificação do livro;
- Desenvolvimento de testbench para validação funcional;
- Execução de simulações que comprovem o correto funcionamento do sistema;
- Documentação clara das decisões de projeto (ex.: políticas adotadas, simplificações, hipóteses).

4. Entrega

A entrega deverá ser realizada via Canvas, na data estipulada, contendo:

4.1 Relatório (formato PDF)

O relatório deve incluir obrigatoriamente:

- Introdução ao tema;

- Descrição do desenvolvimento (arquitetura, decisões de projeto, dificuldades encontradas);
- Resultados obtidos (simulações, validação funcional, testes realizados);
- Conclusão;
- Seção específica sobre uso de IA (ver Seção 8).

4.2 Repositório no GitHub (público)

O relatório deve conter um link para um repositório público que inclua:

- Código-fonte completo do projeto em SystemVerilog;
- Testbenches utilizados;
- Scripts de simulação (se aplicável);
- Arquivo README.md contendo:
 - descrição do projeto;
 - dependências necessárias;
 - instruções detalhadas para compilação e simulação em ambiente Linux.

Observação: A escolha do simulador (ex.: XSIM, ModelSim, Questa, Verilator, etc.) é de responsabilidade do grupo.

5. Organização

- O trabalho poderá ser realizado em grupos de até 5 alunos;
- Todos os integrantes devem contribuir ativamente para o desenvolvimento do projeto.

6. Avaliação

A nota total do trabalho será de 10 pontos, distribuídos da seguinte forma:

- Relatório (5 pontos):
 - Clareza e organização do texto;
 - Qualidade técnica das explicações;
 - Análise dos resultados;
 - Coerência das conclusões.
- Implementação (5 pontos):
 - Correção funcional do controlador de cache;
 - Qualidade do código (organização, modularidade, legibilidade);
 - Cobertura e qualidade dos testes;
 - Estrutura do repositório.

Critérios adicionais:

- A participação individual será avaliada por meio do histórico de *commits* no GitHub;
- Trabalhos incompletos (apenas relatório ou apenas código) não serão aceitos, resultando em nota 0.

7. Testes Mínimos Obrigatórios

Para garantir a validação funcional do controlador de cache, o projeto deve incluir, no mínimo, os seguintes cenários de teste:

7.1 Testes de Leitura (*Read Path*)

- Acesso com cache hit (dados já presentes na cache);
- Acesso com cache miss, seguido de carregamento da memória principal;
- Verificação da atualização correta dos bits de controle (valid, tag).

7.2 Testes de Escrita (*Write Path*)

- Escrita com hit (atualização direta na cache);
- Escrita com miss (comportamento conforme política adotada: write-allocate ou no-write-allocate);
- Validação da política de escrita:
 - write-through, ou
 - write-back (incluindo uso do bit *dirty*).

7.3 Testes de Substituição

- Preenchimento completo da cache e substituição de blocos;
- Validação da política de substituição (ex.: LRU, FIFO ou outra adotada);
- Verificação de escrita em memória em caso de bloco *dirty* (write-back).

7.4 Testes de Consistência

- Sequências de operações leitura/escrita que validem coerência dos dados;
- Acessos repetidos ao mesmo endereço;
- Acessos a diferentes endereços que mapeiam para o mesmo índice (conflitos).

7.5 Testes de Casos Limite

- Acessos a endereços extremos;
- Inicialização da cache (estado vazio);
- Verificação de comportamento com cache completamente inválida.

Observação: Todos os testes devem ser automatizados via *testbench* e acompanhados de evidências no relatório (ex.: *waveforms*, logs ou tabelas de resultados).

8. Uso de Inteligência Artificial (IA)

O uso de ferramentas de Inteligência Artificial (IA) é permitido neste trabalho, desde que respeitadas as seguintes diretrizes:

8.1 Transparência

- Todos os prompts utilizados devem ser incluídos no relatório;
- Deve-se explicitar como a IA foi utilizada (ex.: geração de código, explicações, depuração, testes).

8.2 Modelo de Desenvolvimento

Os prompts devem seguir o modelo de desenvolvimento baseado em especificação, conhecido como:

Spec-Driven Development (SDD)

Isso implica que os prompts devem:

- Definir claramente o problema a ser resolvido;
- Especificar requisitos funcionais e não funcionais;
- Indicar restrições de implementação (ex.: linguagem, interfaces, timing);
- Evitar solicitações vagas ou genéricas.

8.3 Responsabilidade Técnica

- O grupo é integralmente responsável pelo código entregue;
- O uso de IA não substitui o entendimento do projeto;
- O aluno deve ser capaz de explicar e defender todas as decisões implementadas.

8.4 Penalidades

- A ausência dos prompts no relatório será considerada não conformidade;
- Uso de IA sem documentação adequada poderá resultar em penalização na nota do relatório.